

내용 정리

OOP 4대 원칙

캡슐화: 외부노출을 최소화한다.

상속: 공통 속성/기능을 부모에 두고 자식이 물려받음

상속보다 합성이 권장되는 이유는?

우선 상속은 부모의 모든 유전자를 강제로 물려 받는 것을 말한다.

합성은 부모가 가진 좋은 부분만 배치하는 것이다.

금붕어는 어류다 했을 때 절대 변하지 않는 논리적 계층 같은 경우는 상속을 사용하고

자동차는 엔진을 가지고 있다 했을 때 엔진이 전체는 아니니깐 합성

합성 상속이 이해가 안가는거 같습니다.

특히 코드쪽에서

오버로딩 : 과적이라고 생각하면 되는데 제가 이해하기로는 일단 여러 개의 메서드를 만들어놓고

액션이 들어왔을 때 그 중 하나의 메서드를 실행하는 거라고 이해했습니다. (정적 다형성)

if 와 다른 점은 처음 도입부부터 다른 곳으로 보내고 메서드 안에서 if문을 쓰는 것이 다릅니다.

오버라이딩 : 부모클래스에서 상속을 받은 상태에서 자식이 재정의하는 것으로 이해했습니다.

(동적 다형성)

추상클래스와 인터페이스

추상클래스 : "공통 구현 + 일부 추상"/인터페이스는 "규격만"

SOLID

S -Single Responsibility(단일 책임)

제가 이해한 바로는 단일 책임은 각 모듈별로 만들어 놓은 기능만 수정한다.

제 코드는 생각해보겠습니다.

O - open/closed(확장에는 열려, 수정에는 닫혀)

새 구현체만 추가하면 됨

L - Liskov Substitution (자식은 부모를 대체할 수 있어야)

I - Interface Segregation(인터페이스 분리)

D - Dependency Inversion(의존성 역전)

스프링 주입 방식 비교

필드 주입

생성자 주입

1. 불변성 : final 필드 사용 가능 -> 런타임 의존성 변경 불가
2. 테스트 용이성: Spring컨텍스트 없이 new OrderService(mockPayment)로 단위 테스트 가능(필드 주입은 리플렉션 필요)
3. 순환 의존성 조기 발견: 컴파일/기동 시점에 바로 에러 -> 필드 주입은 런타임에서야 발견
4. 명시성: 생성자만 보면 의존성이 한눈에 파악됨

단위 테스트할 때의 차이가 이해가 가지 않습니다.

도구 mockito가 정확히 모르겠습니다.

순환 의존성은 서비스A클래스와 서비스B클래스를 서로 필요로 해서 설계가 잘못된 경우 책임 재분리를 합니다.

Controller -> Service -> Repository 레이어 심화

Controller에서 해야 할 일 : HTTP 요청,응답,변환,입력검증, 인증체크

Service 에서 해야 할 일 : 비즈니스 로직, 트랜잭션 경계, 도메인 규칙

Repository DB 접근만(CRUD)